

4-DAY HANDS-ON PROGRAMME · DAY 2 OF 4

Agentic AI & Agentic Coding

Day 2 — Agentic Coding with Claude Code

Real greenfield and brownfield work driven through an agent — with a review discipline that catches what it gets wrong, and honest measurement of what you gained.

Jai Singh

AI Enablement Consultant & Corporate IT Trainer



Yesterday you built the loop. Today you drive one.



Day 1 — you wrote it

A hand-written loop: three tools, a step cap, a budget, a trace, one branch per stop reason, and a workspace boundary enforced in the tool.



Day 2 — someone else wrote it

Claude Code is that loop with a production tool registry, a permission system, context management, hooks and checkpoints around it. Your job moves from writing the loop to controlling it.

What carries over — and it is most of it

- The context window still fills, and everything is still resent on every step — so what you put in it is a cost decision
- Stopping conditions still matter — they are just spelled differently: --max-turns, a permission mode, a scoped task, and a diff you actually read
- Tool descriptions are still prompts — today they are your CLAUDE.md, your rules, your commands and your skills
- Tool output is still untrusted input — today it is your repository, your dependencies and your ticket text
- A trace is still the only way to debug — today it is the transcript, the diff and the review note

Day 2 at a glance

09:30		Recap & orientation	Four generations of assistance; where Claude Code sits architecturally
10:15		Claude Code fundamentals	Plan and permission modes, context, sessions, checkpoints, headless
11:15		Break	15 min
11:30		Grounding Claude Code in your codebase	CLAUDE.md, rules, skills, subagents, hooks
12:15		Lab 2 — Greenfield build (starts)	A written spec to a running, tested service — agent-driven
13:00		Lunch	45 min
13:45		Lab 2 finish · Lab 3 — Brownfield	Comprehension, characterisation tests, triage, fix, review note
15:15		Break	15 min
15:30		Lab 3b — Grounding clinic	Your first real CLAUDE.md, two commands and one hook — measured
16:00		Portability, discipline & measurement	What transfers, review discipline, anti-patterns, IP, metrics
16:45		Debrief & checkpoint	One reviewed merged change, review note, working agreement

Timed labs: 2 h 45 m. The two taught sessions run at the keyboard — you do every step in your own repository — so hands-on today is closer to five hours.

What you leave with



One merged change

A reviewed, agent-assisted change on a branch, with tests, that you can walk someone through line by line.



A review note

What you accepted, what you changed, what you rejected, and what a human still has to verify — each with the evidence behind it.



Grounding that works

A CLAUDE.md that earns every line, two commands your team would actually use, and one hook that enforces rather than asks.



A trustworthy explanation

A method for getting an account of unfamiliar code and then checking it — because an unchecked explanation is a hypothesis.



Characterisation tests

The guardrail that tells a refactor from a regression, written before anything is changed.



An honest number

Your own estimate versus your actual time, including the tasks that would have been faster by hand.

SESSION 1

Recap & Orientation

Four generations of coding assistance, and where this tool actually sits

09:30 – 10:15



Four generations of coding assistance



1 · Completion

Predicts the next few lines from the open file

You keep the whole plan in your head; the tool saves keystrokes



2 · Chat

Answers questions in a side panel; you copy the answer across

It sees only what you paste, so its answers stop at the edge of that window



3 · Agent mode

Reads and writes files, runs commands, iterates on its own output

It can now do the wrong thing to many files at once



4 · Headless

Runs unattended, in a pipeline or a scheduled job, with no one watching any step

Correctness has to be enforced by gates, not by a person

The discontinuity is between 2 and 3

Generations 1 and 2 produce suggestions you accept one at a time. Generation 3 takes actions. That is why everything from here on — permissions, scoping, diffs, review notes, hooks — is about controlling actions rather than improving suggestions.

Where Claude Code sits: terminal-native, not an editor plug-in



It is a terminal agent with an IDE surface, not an IDE feature

- The extension shows diffs, selections and the open file. The agent, the permissions and the configuration are the same in either place — so what you learn in the terminal is what runs in CI.

The repository is the unit of configuration

- What lives in `.claude/` is committed, reviewed and shared like code — apart from `settings.local.json`, which stays yours and gitignored. Grounding one machine helps one person; grounding the repository helps the team and survives onboarding.

On Bedrock, Claude Code uses the Invoke API

- Not the Converse API. It resolves models through cross-region inference profiles, and on our account through per-participant application inference profiles so spend is attributable to a person.

What actually changes when the agent has your repository

	Chat assistant	Agentic coding tool	What that costs you
Input	What you paste	Whatever it decides to read	Context is now a budget you manage, not a box you fill
Output	A suggestion	An edit already on disk	Review moves from 'before I paste' to 'before I commit'
Blast radius	One file, one paste	Every file it can reach	Scope has to be decided up front, not discovered in the diff
Error mode	A wrong answer you ignore	A confident change that passes tests	Tests stop being sufficient evidence on their own
Trust boundary	Your own words	Repo text, tickets, dependencies, tool output	Untrusted input is now inside the loop — Day 3 covers this properly

The whole of today is a response to row 3 and row 4: keep the blast radius small enough to review, and make the evidence good enough to trust.

SESSION 2

Claude Code Fundamentals

The core loop, the permission system, the context window, and running it without a human

10:15 – 11:15



The core loop: plan → act → diff → review



The failure mode is skipping step 3. Everything else on this slide is cheap; reading the diff is the part that costs attention, and it is the only part that catches a confident, wrong, test-passing change.

Plan mode: think before it edits

Shift+Tab cycles the mode, or start in it with `claude --permission-mode plan`

What plan mode does

- Claude reads files and runs read-only commands to explore
- It does not edit your source files
- It comes back with a plan you approve, edit or reject
- The approved plan is re-injected from disk after a compaction, so it survives a long session

When it earns its cost

- Unfamiliar code — the plan tells you what it believes before it acts on the belief
- Anything touching more than two or three files
- A change you would have asked a colleague to sketch first
- Not needed for a typo, a rename, or a one-line fix

Read a plan for these four things



Scope

Does it change only what you asked about? A plan that mentions files you did not expect is telling you something.



Assumptions

Which of its statements about the code are claims rather than facts? Those are what to check first.



Evidence

What will prove this worked? If the plan has no test, you are about to accept a change on its word.



Omissions

What is not in the plan that should be — the migration, the caller, the error path, the rollback?

Permission modes: what it may do without asking

Mode	Behaviour	Use it when
default (Manual)	Prompts on first use of each tool	Learning a repository, or any change you have not scoped yet
plan	Reads and runs read-only commands; does not edit your source files	Before any non-trivial change — the default opening move
acceptEdits	Accepts file edits and common filesystem commands without prompting	A scoped slice you have already planned, in a branch you can throw away
auto	Auto-approves with background safety checks on each action	Longer runs where you will still read the whole diff afterwards
dontAsk	Auto-denies anything that would prompt; allow-listed tools still run	Unattended and CI runs — nobody is there to answer a prompt
bypassPermissions	Skips prompts almost entirely	Only inside a container or VM you can destroy — not on your laptop

Set the mode a session starts in with `defaultMode` in `.claude/settings.json`, or `--permission-mode` on the command line. Organisations can block `auto` and `bypassPermissions` outright with `permissions.disableAutoMode` and `permissions.disableBypassPermissionsMode` in managed settings.

Tool allow-lists: the rules live in the repository

THIS REPOSITORY'S ACTUAL RULES

```
// .claude/settings.json - committed, reviewed like code
{
  "permissions": {
    "allow": [
      "Bash(mvn -q test*)",
      "Bash(npm run *)",
      "Bash(git status)",
      "Bash(git diff*)",
      "Read(**)"
    ],
    "ask": [ "Bash(git push*)" ],
    "deny": [
      "Read(./env)",
      "Read(./secrets/**)",
      "Bash(aws iam *)"
    ]
  }
}
```

Rule	Matches
Bash(npm run build)	That exact command, nothing more
Bash(npm run *)	Any command starting npm run — the space before * matters
Read(./env)	Reading .env in the project root
Edit(src/**)	Files under ./src only; use **/src/** for any depth
WebFetch(domain:github.com)	Fetches to that host

Deny wins — over the mode, and over a hook

- A deny rule blocks the call whatever the mode, and whatever a PreToolUse hook returns. Deny and ask rules are also the ones the trust dialog does not gate — they only restrict, so they apply from the first run.

But a Bash rule is not a filesystem boundary

- Read(./env) stops the file tools. It does not stop a script that an allowed command runs — Bash(npm run *) permits any script the repository defines. For a boundary that holds, use sandboxing, not rules.

Allow rules are per-repository trust

- "Yes, and don't ask again" writes to settings.local.json — yours, gitignored.

Context: what is already in the window before you type

System prompt and tool definitions	Always. You never see it.
CLAUDE.md files — managed, user, project, local	Every session, concatenated from the root down
Unscoped .claude/rules/*.md	Every session, same priority as the project CLAUDE.md
Auto memory (MEMORY.md)	First 200 lines or 25 KB, whichever comes first
Skill names and one-line descriptions	Names only — a skill's body loads when it is invoked
MCP tool names	Schemas load on demand unless you change the default
Environment and git status	Working directory, platform, branch, recent commits



/context

Shows the breakdown for the live session, including which memory files actually loaded. If your CLAUDE.md is not listed there, the agent cannot see it — that is the first thing to check when it ignores an instruction.

The economics have not changed

Every step resends the whole window. Prompt caching makes the stable part cheaper on repeat requests — not free, and it still occupies the window. A 200-line CLAUDE.md is not a one-off cost. Target under 200 lines per file, and move anything conditional into a path-scoped rule or a skill.

Compact, clear, or start again



`/compact`

Replaces the conversation with a structured summary. A focus — `/compact focus` on the validation rules — steers what it keeps. Guidance, not a guarantee: check the decisions you cared about survived.



`/autocompact 500k`

Sets how full the window gets before the automatic pass runs. Automatic compaction works the same way — a full window does not end your session.



`/clear`

Throws the conversation away and starts clean. The right move between unrelated tasks: old conversation crowds out the files you need next.

What survives a compaction — and what does not

Survives — re-injected from disk	Summarised away
Project-root CLAUDE.md and unscoped rules	Anything you only said in conversation
Auto memory, and the plan from plan mode	Rules with paths: frontmatter, until a matching file is read again
Up to five recently modified files it read	Nested CLAUDE.md, until it reads that subdirectory again
Bodies of skills you invoked (capped per skill)	The startup skill listing — only invoked skills come back

Sessions, resuming, and rewinding a mistake

SESSIONS AND CHECKPOINTS

```
claude          # new session in this directory
claude --continue # resume the most recent one
here
claude --resume <id> # resume a specific session by
id

/rewind          # or press Esc twice on an
empty prompt
  Restore code and conversation
  Restore conversation    (keep the code)
  Restore code            (keep the conversation)
  Summarize from here / up to here
```



How checkpoints work

- Every prompt that starts a turn captures the state of your code first
- The last 100 checkpoints in a session are kept, and they are saved with the conversation, so `/rewind` still works after you resume
- Restoring code and restoring the conversation are separate choices — you can keep what you learned and throw away what it wrote



What rewind does NOT restore — know this before you rely on it

- Files changed by Bash commands — `rm`, `mv`, `cp`, a script, a build step. Only edits made with the file-editing tools are tracked.
- Edits made by a subagent, in almost every case. Use `git` for those.
- Changes you or another session made outside this one, and symlinked or hard-linked paths.

Headless: the same agent, with nobody watching

claude -p

```
timeout 600 claude -p "Review the diff against main for error handling" \
  --bare \
  --append-system-prompt-file .claude/review-standard.md \
  --permission-mode dontAsk \
  --tools "Read,Grep,Bash" \
  --allowedTools "Read,Grep,Bash(git diff *)" \
  --max-turns 12 \
  --output-format json > out.json
status=$? # a pipe into jq would hide this

# 1. did the run itself work?
[ "$status" -eq 0 ] || exit "$status"
jq -e '.is_error == false' out.json >/dev/null || exit 1

# 2. only now, did the REVIEW pass? that is your own rule
jq -r '.result' out.json
```

--bare

- Skips auto-discovery of hooks, skills, commands, subagents, MCP servers, auto memory and CLAUDE.md. In CI you want the same result on every machine — not whatever a teammate has in their home directory.

--max-turns, and an outer timeout

- A step cap is not optional because the agent is good. --max-turns bounds the loop; the timeout bounds everything else.

--tools and --allowedTools

- --tools is what exists at all; --allowedTools is what runs without prompting. Different knobs, and CI wants both.

Ran ≠ passed

- Exit 0 and is_error false mean the run worked. Whether the review passed is your rule, applied to .result — a clean run can still report blocking findings.

Today you meet it; Day 4 puts it on the merge path.

A headless run has no one to catch a bad change, so the guardrails move into the pipeline: a fixed allow-list, a permission mode that denies rather than waits, a turn limit, an outer timeout, and a gate that fails the build rather than a human who notices. Note the trade — --bare drops your CLAUDE.md, hooks and commands, so pass the review standard back in explicitly. Without --bare, a -p run in a repository loads and runs that repository's hooks and MCP servers, trusted or not.

SESSION 3

Grounding Claude Code in Your Codebase

Five layers, from a polite request to a gate the agent cannot talk its way past

11:30 – 12:15



Five layers — and the difference between asking and enforcing



CLAUDE.md

Always in context

Facts it needs in every session: build commands, conventions, traps



.claude/rules/

In context, or on match

The same, split by topic — and scoped to paths so it loads only when relevant



Skills

Loads when invoked

A repeated procedure, with its supporting files. Name and description only until used



Subagents

Its own context window

Narrow work delegated to a focused configuration — reviewing, searching, planning



Hooks

Runs on the event

A shell command on a lifecycle event. Exit 2 blocks a matching call. Not a request

context the agent reads → behaviour the agent chooses → rules the agent cannot choose around

Choosing the layer: if getting it wrong is annoying, write it in CLAUDE.md. If getting it wrong is unacceptable — a secret committed, a skipped test gate — it belongs in a hook. Asking politely works most of the time; a correctly configured PreToolUse hook blocks the calls its matcher covers, every time. Neither is a merge gate: for anything mandatory, keep the CI check as well.

CLAUDE.md: the memory hierarchy

Scope	Location	What belongs there	Shared with
Managed policy	/Library/Application Support/ClaudeCode/CLAUDE.md · /etc/claude-code/CLAUDE.md	Organisation-wide standards and compliance reminders. Cannot be excluded by a user setting.	Everyone, every repo
User	~/.claude/CLAUDE.md	Your own preferences across all projects. Not project facts.	Just you
Project	./CLAUDE.md or ~/.claude/CLAUDE.md	Build and test commands, conventions, architecture decisions, traps. This is the one that matters today.	The team, via git
Local	./CLAUDE.local.md (gitignored)	Your sandbox URLs, your test data, your personal notes on this repo.	Just you, this repo

Loading order

- Concatenated from the filesystem root down to your working directory, so the most specific instruction is read last. Files in subdirectories load on demand.

Two commands worth knowing

- /init drafts one from the codebase — a starting point, not a finished file. /memory lists and opens them. /context says which ones actually loaded.

What earns a line in CLAUDE.md



Include

- How to build and test — the exact command, including the flag that is easy to get wrong
- Conventions not discoverable from the code in one read
- Things that will actively mislead: a dead directory, a legacy module nobody should copy, a naming rule with an exception
- Decisions with a reason attached — "standard library only, because twenty-two laptops behind a proxy"



Leave out

- Anything the agent can read from the code in a few seconds — the directory tree, the dependency list
- Your architecture diagram
- Aspirations. Write what is true today, not what you wish were true
- Anything conditional on one part of the codebase — that is a path-scoped rule, or a skill



The test for every line: would the agent have got this wrong without it?

If not, delete it. A long CLAUDE.md is not a better one — it is context you pay for on every single request, and past about 200 lines adherence falls off rather than improves.

A real one — from the repository in front of you

airamatrix-agentic-labs/CLAUDE.md — excerpt

Conventions

- **Python and Node labs use the standard library only.** No pip install, no npm install. Twenty-two laptops behind a corporate proxy is not the place to discover a dependency problem.
- **Java builds through the root POM.** Always -am, or the labkit dependency will not resolve:
mvn -q test -pl day1-foundations/lab1-bare-metal-loop/java -am
- **Every lab has offline tests that always run, and live tests gated on LAB_LIVE=1.** Live tests call the model and cost money.

When editing labs

- A starter must **fail** its own tests until implemented. If a change makes the starter pass, the lab is broken.

What is deliberately not here

Model IDs, gateway URLs and budgets are **not** hardcoded anywhere in lab code. Changing the model for the whole room is a gateway-side edit, not 22 file edits.

Every line passes the test

- None of it is discoverable in one read. The -am flag, the proxy constraint and the LAB_LIVE gate are all things an agent gets wrong on its first attempt — repeatedly. Note the scope, too: the no-install rule is about the lab exercises, and a line that does not say where it applies will be over-applied.

Reasons, not just rules

- "No npm install" alone invites a workaround. With the reason attached, the agent applies the same judgement to a case you did not anticipate.

A section for traps

- "A starter must fail its own tests" is counter-intuitive. Without it, a helpful agent fixes the lab and breaks the exercise.

It is under 60 lines

- Read it in full during Lab 3b. It is deliberately short, and it is the model for the one you will write.

.claude/rules/ — instructions that load only when they apply

ONE TOPIC PER FILE

```
your-project/
└── .claude/
    ├── CLAUDE.md          # always loaded
    └── rules/
        ├── code-style.md  # no paths: -> always loaded
        ├── api-design.md  # paths: -> loaded on match
        └── testing.md
```

Why this matters more than it looks

- A rule with no paths: costs context on every request, all day. A path-scoped rule costs nothing until the agent opens a file it applies to.

The catch, after a compaction

- Unscoped rules are re-injected from disk. Path-scoped rules are not — they come back when the agent next reads a matching file. If a rule must never lapse, drop the paths: or move it to the project CLAUDE.md.

Shared across repositories

- Symlink a shared directory into .claude/rules/, or keep personal rules in ~/.claude/rules/. Project rules win over user rules.

PATH-SCOPED — LOADS WHEN A MATCHING FILE IS READ

```
---
paths:
  - "src/api/**/*.ts"
  - "src/**/*.{ts,tsx}"
---

# API development rules

- Every endpoint validates its input before anything else
- Use the standard error response shape
- Include the OpenAPI comment block
```

Rules are for what is true about a part of the codebase. A procedure — the steps to do a recurring task — belongs in a skill instead.

Skills: a repeated task, packaged with its files

`.claude/skills/<name>/SKILL.md`

```
.claude/skills/review-diff/
├── SKILL.md           # required; the directory name is the command
├── checklist.md       # supporting files travel with it
└── scripts/lint.sh
```

```
---
name: review-diff
description: Review the working-tree diff the way Day 2 asks you to
allowed-tools: Read, Grep, Bash(git status *), Bash(git diff *)
---
```

The change set

```
!`git status --short --untracked-files=all`
!`git diff`
!`git diff --staged`
```

Instructions

Read every file listed as untracked (??) in full - a new file appears in no diff at all. Say which change set you reviewed, then assess: scope, correctness, tests, security, what was removed, and what a human must verify. End with accept / with changes / reject.

Two ways it runs

- You type `/review-diff`. Or the agent invokes it itself when your request matches the description — which is why the description is the most important line in the file.

It costs nothing until used

- Only names and descriptions are in context at startup. The body loads on invocation. Set `disable-model-invocation: true` and it costs nothing at all until you ask for it.

Commands are now skills

- `.claude/commands/review-diff.md` still works and still gives you `/review-diff`. The skill form adds supporting files, frontmatter and the ability for the agent to reach for it unprompted.

`--untracked-files=all` is not optional

- Plain git status collapses a whole new directory into one line: `?? src/`. On a greenfield task that hides the entire implementation.

Put the important part first

- After a compaction, an invoked skill's body is re-injected but truncated from the top down.

Building the team's command library

/review-diff — IN THIS REPOSITORY TODAY

****First, establish what you are reviewing.**** `git diff` alone shows tracked, unstaged changes only. Run all three:

```
git status --short --untracked-files=all
git diff
git diff --staged
```

Without `--untracked-files=all`, git collapses a whole new directory into one line, "?? src/", and you review none of it. Read every file it lists, in full. State in one line which change set you reviewed. For a committed branch, use ``git diff <base>...HEAD`` instead.

Then assess, in this order:

1. ****Scope.**** Does it do one thing? Two things is two changes.
2. ****Correctness.**** What would break? Name the input or state.
3. ****Tests.**** Actually covered, or only apparently? A test that passes against a stub is not coverage.
4. ****Security.**** Any secret, path escaping its sandbox, new dependency?
5. ****What was removed.**** A dropped check breaks no test that never covered it, so deletions need reading, not running.
6. ****What a human must verify.**** List it explicitly.

End with ****accept****, ****with changes**** or ****reject****, and one sentence.

It establishes scope before it reviews

- An empty unstaged diff does not mean there is nothing to review. This is the single most common way an agent-assisted review reviews nothing at all.

It fixes an order

- Left to itself the agent leads with style; here it cannot reach style before correctness and coverage.

It forbids padding

- "Report nothing rather than padding" is the single most valuable instruction in a review prompt. Three invented findings are worse than none.

Good first candidates

- Run the tests the way CI does. Explain a module. Write the migration note. You will write two of these in Lab 3b.

A command is how a senior engineer's review standard becomes something every member of the team runs, identically, without being in the room.

Subagents: narrow work, its own context window

`.claude/agents/reviewer.md` — in this repository

```

---
name: reviewer
description: Second-opinion reviewer. Use to check work another
            agent produced, before a human sees it.
tools: Read, Grep, Glob
model: inherit
---

You did not write this code. That is your advantage: you have no
investment in it being correct.

1. Find what is wrong, not what is stylistically different.
2. For each finding give a concrete failure: the input, the state,
   and the resulting wrong behaviour. A finding you cannot make
   concrete is a hunch - label it as one or drop it.
3. Check the tests exercise the claim they are named after.
4. Report nothing rather than padding.

Never edit files. You review; someone else decides.

```

Remember from the checkpoint slide: edits made by a subagent are not restored by /rewind. Use git.

A separate window is the feature

- It gets your CLAUDE.md and the task, not your conversation. Everything you argued with it about is gone — which is exactly what you want from a second opinion.

tools: narrows the door — mind its width

- This one has no Write, no Edit and no Bash, so it cannot change the code it is judging. Leave Bash in and it can: a shell command writes files too, subject only to your permission rules. "Never edit files" in the body is an instruction, not a boundary.

model: inherit, not an alias

- sonnet resolves to Claude Code's built-in Bedrock default, which your IAM user cannot invoke. inherit keeps the profile ARN you pinned.

When it is worth the complexity

- Reviewing, searching a large codebase, and any long read-only investigation whose intermediate steps you do not want in your own window.

When it is not

- Anything you would have to re-explain. The handover costs more than the work. Day 4 builds the evaluator/reviewer pair properly.

Hooks: enforced, not requested

`.claude/hooks/block-secrets.sh` — ILLUSTRATIVE DETECTOR

```
#!/usr/bin/env bash
set -uo pipefail
payload=$(cat)          # the hook's JSON arrives on STDIN

# Scan the PROPOSED content, never the whole payload: an Edit
# carries old_string too, so grepping the lot blocks a REMOVAL.
proposed=$(printf '%s' "$payload" | python3 -c '
import json,sys
d=json.load(sys.stdin); t=d.get("tool_name"); i=d.get("tool_input",{})
print(i.get("content","") if t=="Write" else i.get("new_string",""))
')

if printf '%s' "$proposed" | grep -qE \
  '(sk-(ant|aira)-?[A-Za-z0-9_-]{20,}|(AKIA|ASIA)[A-Z0-9]{16})'
then
  echo "Blocked: that would write a credential." >&2
  exit 2          # blocks THIS call, whatever the JSON says
fi
exit 0
```

A SCRIPT IS NOT A HOOK UNTIL IT IS REGISTERED — `.claude/settings.json`

```
{ "hooks": { "PreToolUse": [ {
  "matcher": "Write|Edit",
  "hooks": [ { "type": "command",
    "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/block-secrets.sh" } ]
} ] }
```

Covers Write and Edit only — not a secret written by a shell command — and matches two access-key prefixes (AKIA, ASIA) plus two token shapes. Teaching example, not the control: the repository check is a maintained scanner in CI.

Event	Fires
PreToolUse	Before a call — the one that blocks
PostToolUse	After it succeeds — format, lint
UserPromptSubmit	Before your prompt is processed
SessionStart	Session begins or resumes
PreCompact	Before compaction
Stop	Claude has finished responding

Read stdin, scan the new text

- Both halves matter. Miss the first and the hook silently allows everything; miss the second and it blocks you from removing a key. A common mistake, and one the tests catch in a second.

Exit codes, for a PreToolUse command hook

- 0 — proceed. 2 — block, and stderr goes back to the agent as the reason. Anything else — non-blocking, and the action proceeds. Other events read these differently.

Test the failure path, not just the block

- A script that is missing, not executable, or times out does not block anything. PostToolUse runs after the fact and cannot undo it.

SESSION 4

Lab 2 — Greenfield Build

A written specification to a running, tested service — agent-driven, inside 60 minutes

12:15 – 13:00 · 13:45 – 14:00



What you are building: the Slide Intake Service

A small HTTP service that accepts slide-analysis jobs, validates them, and reports status. Nothing is scaffolded for you — scoping the work so the agent can finish it is half the exercise.

Method	Path	Behaviour
POST	/jobs	Submit a job. Returns 201 with the created job.
GET	/jobs/{id}	Return the job, or 404.
GET	/jobs?status=queued	List jobs, optionally filtered by status.
POST	/jobs/{id}/cancel	Cancel a queued job. 409 if it is already completed.

THE JOB

```
{
  "id": "generated, opaque",
  "accountId": "ACC-1001",
  "slideCount": 250,
  "priority": "normal",
  "status": "queued",
  "submittedAt": "2026-09-19T09:30:00+05:30"
}
```

Status transitions



queued → cancelled is also allowed. Any other transition is a 409, including one for a job id that does not exist. · Nothing external drives running and completed: expose them as an internal method the tests call, so every transition is reachable without a worker.

Non-functional

- Tests for every validation rule and every rejected transition
- No database — in memory is fine
- Java/Spring Boot or TypeScript/Node
- Front-end teams: build the client, not a second back end — against the trainer's reference API. Submit a job, show field-level errors for a rejection that fails three rules at once, poll status

The five validation rules — where agents cut corners

1 accountId must match ACC- followed by exactly four digits

Watch for a regex that is not anchored — ACC-12345 and XACC-1234 both slip through

2 slideCount must be between 1 and 5000 inclusive

Inclusive. Ask what happens at 0, 1, 5000, 5001, -1, missing, and a string

3 priority must be one of low, normal, rush

A closed set is an enum, not a free-text field with a comment

4 A rush job above 1000 slides is rejected — rush capacity is limited, and the error must say so specifically

Cross-field. Probe it directly: priority=rush, slideCount=1001 must fail; rush at 1000 must pass

5 Rejections return 400 naming every failed field, not just the first

Probe it with one payload that breaks three rules at once and count the fields named in the body

Rules 4 and 5 are the ones to check specifically. They are also the two a reviewer skims past, because the tests are green and the happy path works.

Scoping: one intent per change



Not a slice

- "Build the Slide Intake Service"
- "Add the endpoints and the tests and wire up the error handling"
- "Make it production-ready"



A slice

- "Add the Job model and its five validation rules, with tests"
- "Add POST /jobs returning 201, using the existing validator"
- "Add the cancel transition and the 409 cases"

Three questions that size a slice correctly



Can I read the diff?

A diff you cannot hold in your head is too big. One screen is a good heuristic, not a law — a cohesive change with its tests may fairly run longer.



Can I name what done means?

If you cannot say what proves it worked before you ask, the agent cannot either — and it will decide for you.



Can I throw it away?

One slice should be revertible on its own. Two intents in one commit means neither can be backed out cleanly.

Greenfield build — spec to running service

OBJECTIVE

Take a written specification to a running, tested service, agent-driven, with every diff reviewed before the next one starts.

TIME 60 min

BEFORE YOU START

- Fill in metrics/baseline-estimate.md: how long by hand, the riskiest part, how many tests you would write
- An estimate written afterwards is worthless — this is the only honest comparison you get
- Front-end teams: get the trainer's reference API host:port now, and use your own Angular scaffold if you have one checked out — otherwise one dependency-free page with fetch

1. Read greenfield/SPEC.md. Do not paste it wholesale — decide the slices yourself
2. Ask for a plan. Read it. Push back where you disagree — and if you agree with all of it, say what you checked
3. Slice 1: the model and the five validation rules, with tests. Boundaries: slideCount 0, 1, 5000, 5001, -1, missing, a string
4. Read the whole diff. Then /review-diff
5. Slice 2: the endpoints. Slice 3: the transitions and their 409s
6. Probe rule 4: rush at 1001 must fail, rush at 1000 must pass. Probe rule 5: one payload breaking three rules must name all three
7. Fill in the second half of baseline-estimate.md while it is still fresh

Done when: the service runs, every validation rule and every rejected transition has a test, and no diff went unread before the next one started.

SESSION 5

Lab 3 — Brownfield and the Grounding Clinic

Comprehension, characterisation tests, triage, then change — the highest-value session for a team with an existing product

14:00 – 15:15 · 15:30 – 16:00



The situation: legacy-svc



legacy-svc bills slide analysis. Written for one customer in 2021 and extended since. It has no production tests — two starter characterisation tests are provided, the rest are yours. The nightly invoice run is crashing, roughly one account in forty.

What you have been given



The code

BillingEngine.java — tiers, volume discounts, rush handling, prepaid credits, tax and rounding, in one method.



A stack trace

NullPointerException at calculateInvoice line 47, reached through calc(...) from the nightly batch job.



A defect report

INT-4488: the crash, plus two more complaints that arrived the same week. Nobody has confirmed they are the same bug.

If a sanitised copy of your own repository is available, use that instead — legacy-svc exists so the lab runs regardless. The method is the point, not this particular billing engine.

Step 1 — Comprehension you can actually trust (10 min)

```
/explain-module day2-agentic-coding/brownfield/legacy-svc  
  
# or simply  
"Explain BillingEngine.calculateInvoice. What does each block do,  
and which inputs reach each branch?"
```



What it is good at

- Reading a long method and naming its parts
- Enumerating the inputs that reach each branch
- Spotting that a variable is computed and never used
- Answering "who calls this?" across the repository



What it will get wrong

- Intent — why the code does this, which is not in the code
- Which behaviours callers actually depend on
- Whether something is a bug or a deliberate, load-bearing quirk
- It states all four as confidently as the first list



Then verify it. Pick two claims and check them against the code yourself.

An explanation you have not checked is a hypothesis, and this is the step people skip — the explanation is fluent, it sounds like a colleague, and the next forty minutes are built on it.

Step 2 — Characterisation tests first (25 min)

A characterisation test pins what the code DOES, not what it should do — bugs included. That is the point. Without it you cannot tell a refactor from a regression.

```
mvn -q test -pl day2-agentic-coding/brownfield/legacy-svc -am

# Two tests are written for you. The rest are yours.
# Ask the agent to enumerate cases - it is good at that -
then
# read every one it produces.
```



The specific danger here

An agent-written characterisation test that asserts the wrong current behaviour is worse than no test at all — because you will trust it, and it will bless the regression you were trying to avoid. Read every one.

- Cover every tier — gold, silver, bronze — and ACC-1099, whose tier was never backfilled
- Cover the boundaries — slideCount at 0, 1, 500, 501, 1000, 1001 — the discount thresholds are where behaviour turns
- Cover rush and prepaid credits — including credits larger than the total, which zeroes the invoice
- Assert what it does today — record the current rounded figure even when it looks wrong to you

Why first, and not after

- You can always check out the old revision and run it — but reconstructing what it produced, case by case, after the fact is slow and easy to get wrong. Capture the baseline first and the comparison is automatic. The tests are not for the fix; they are for everything the fix moves that you did not intend.

This is the transferable part

- Nothing about this step is specific to agents. It is the discipline that makes agent-assisted brownfield work safe at all.

Step 3 — Triage: three symptoms, not all of them real (10 min)

Reported	What to check	Verdict — work this out yourself
The nightly run crashes, about one account in forty	The trace points at line 47, reached through calc(...). Which accounts reach that line with a value the code does not expect?	Reproduces
"Invoices for zero-slide accounts fail"	Write the test. A zero-slide invoice — does it throw, or does it quietly produce a value nobody looks at?	Read the code, then decide
"The rounding is wrong — ₹1,234.567 came through as ₹1,234.00"	Look at the last line before the return. What are the types on each side of that division?	Read the code, then decide



One of the three does not reproduce at all.

Finding that out is worth more than fixing it would have been. A defect report is a set of claims, not a work list — and "we investigated and it does not happen" is a result you can defend.

Step 4 — Fix what is real, without moving what isn't (15 min)



The tests tell you what you moved

If a characterisation test now fails, that is a behaviour change. Was it deliberate? Say so in the review note, or put it back.



There is a second caller

calc(...) is used by an old batch job and INT-4471 says do not delete it. Does your fix change what that caller sees?



Some questions are decisions

The rounding is a money question, not a code question. Decide it deliberately, write down why, and flag it for someone who owns the number.

The question to ask before every brownfield fix

"Who else depends on the behaviour I am about to change?" — the agent cannot answer this from the code alone, because the answer lives in callers it has not read, in batch jobs, in a ticket from 2021, and in a customer's reconciliation spreadsheet.

Step 6 — the review note (5 min)

Accepted

What you kept — and the evidence: the test you read, the case you traced

Changed or rejected

What you corrected or threw away, and what was wrong with it

Still unverified

What a human has to check that no test covers

Refactor second — and the same guardrail for an upgrade

Now that the tests exist, the tidier rewrite the agent offered at the start becomes a reasonable thing to consider — because you can now show what it did and did not move, for the cases you covered.

Step 5 — one behaviour-preserving refactor (10 min)

- Pick one thing only: extract the discount ladder, or name the magic numbers. Not both, and not the whole method.
- Ask for it as a refactor with no behaviour change, and say the characterisation tests are the contract.
- Run them. Green is evidence for the cases you covered, not proof for all of them — so read the diff and the callers too. A red test needs investigating before anything else: a real regression, an environment problem, or a flaky test. If it is a regression, read it — do not re-baseline it.
- For this lab, follow that sequence: comprehension, then characterisation, then fix, then refactor. The order is what makes the last step safe.

Bug reproduction from a stack trace is the same loop run backwards: start from the trace, write the failing test, then fix.

Taught, not timed — the same shape for an upgrade

- 1 Pin what you have** Record the current versions and get the suite green before touching anything
- 2 Ask for the compatibility read** What breaks between these versions, and which of it applies to this code? Verify the claims — this is where agents are most confidently wrong
- 3 One upgrade per change** Separate independent upgrades; keep the compatibility changes a single upgrade requires together with it
- 4 Let the tests find the rest** Behaviour the release notes did not mention is exactly what characterisation tests are for
- 5 Know the way back** The revert has to be one command. If it is not, the slice was too big

Brownfield — comprehension, characterisation, change

OBJECTIVE

Take an untested legacy module from "nobody understands it" to a defensible, tested change — with a written record of what you accepted, changed and rejected.

TIME 75 min · 10 + 25 + 10 + 15 + 10 + 5

THE TRAP

- The agent will offer to rewrite `calculateInvoice` into something much tidier. It will probably be tidier.
- Accepting that before you have characterisation tests is exactly how brownfield refactors go wrong — the single most common failure mode of agentic coding on real systems.

1. Comprehension (10): get an explanation, then verify two of its claims yourself
2. Characterisation tests (25): every tier, every discount boundary, rush, prepaid credits
3. Run them. They pass — they describe today, bugs included
4. Triage (10): the three reported symptoms, against those tests
5. Fix (15): what reproduces. Check `calc(...)` still behaves for its caller
6. Re-run every characterisation test; account for each one that moved
7. One behaviour-preserving refactor, evidenced by the same tests and a read of the diff
8. Review note (5), then take the change through the merge procedure

Done when: you can say which reported symptom was not real, the crash is fixed, the batch-job caller still behaves, and review-note.md exists.

Grounding clinic — write it, then measure it

OBJECTIVE

Write the context that makes an agent useful on your codebase, and prove whether it changed anything.

TIME 30 min

MEASURE BEFORE, NOT AFTER

- Pick a task the agent gets wrong without context: "add an endpoint following our conventions", "write a test for X", "fix this build".
- Run it. Save the output. That is your baseline — and it only exists if you take it first.

1. Target your own repository if you have it checked out, otherwise legacy-svc
2. Run the chosen task cold, on a clean checkout in a fresh session. Save what it produces
3. Write a CLAUDE.md — every line must pass "would it have got this wrong without it?"
4. Add two slash commands your team would actually use
5. Add one hook that enforces something: formatting, or a secret scan
6. Reset to the same commit, start another fresh session, re-run the same task, and compare
7. Delete the lines that earned nothing — but one unchanged run is weak evidence, so keep a line you can argue for

Done when: a CLAUDE.md that earns every line, two commands, one hook that enforces, and a before/after you can show someone.

SESSION 6

Portability, Discipline and Measurement

What transfers to the next tool, what a human must always verify, and what is worth counting

16:00 – 16:45



What transfers between agentic coding tools



Transfers

Task scoping

- One intent per change, a diff you can read, a stated definition of done. Every tool, every vendor.

Repository context files

- The idea of committed, reviewed instructions. The filename differs — CLAUDE.md, AGENTS.md, .cursor/rules — the content barely does.

Review discipline

- Reading the diff, checking the tests exercise their claim, writing down what you rejected.

MCP servers

- An open protocol. A server you build on Day 3 works with any client that speaks it.



Does not transfer

Directory layout and tool-specific syntax

- .claude/ paths, hook event names, permission rule strings. The SKILL.md idea — a folder, instructions, its supporting files — is becoming common ground; the Claude-specific frontmatter is not.

The permission model

- Modes, allow-lists and deny precedence are specific to this tool. Others ask differently, or not at all.

Hooks and their exit codes

- The concept of a deterministic gate transfers. The wiring does not.

Session and checkpoint behaviour

- What is tracked, what survives compaction, what /rewind restores.

Roughly: the discipline transfers, the configuration does not. Budget a day to re-express your grounding in another tool's syntax — and budget nothing to relearn how to scope a task or read a diff.

Review: what a human must always verify

Tests tell you the code does what the tests say. They cannot tell you the tests were the right ones.



Intent

Does this do what was asked, or something adjacent that also passes? The most common agent error is a correct solution to a slightly different problem.



Test quality

Does each test exercise the claim in its name? A test that passes against a stub, or asserts the mock, is not coverage.



Blast radius

What else reads this? Callers, batch jobs, serialised formats, a report someone reconciles by hand.



Security

Any secret, any credential, any path that escapes its sandbox, any new dependency you did not ask for.



What it removed

Deletions are the least-read part of a diff and the most dangerous. A dropped null check passes every existing test.



What no test covers

Every agent-assisted change has some. A review note claiming none is not finished.

Four anti-patterns — and the rubber-stamping trap



Context flooding

Pasting whole files, whole specs, whole logs "so it has everything". It costs on every step, dilutes the instruction that mattered, and buries the constraint you cared about.

Give it the two files and the one rule.



Agent thrash

The same fix attempted five ways in a row, each undoing the last. Usually a sign the goal is under-specified or the error is not what it looks like.

Stop at the second repeat. Read the actual error yourself.



Scope creep in one run

You asked for a validation rule; the diff adds a config layer, a logger and a retry policy. Each is defensible. Together they are unreviewable.

Reject the diff, not the additions. Re-ask, smaller.



Security-blind acceptance

A new dependency, a widened permission, a credential moved into a file "for convenience", a disabled check that was failing.

Read deletions and dependency changes first, not last.



The rubber-stamping trap: the fifth good diff in a row is what makes you stop reading the sixth. It is not laziness — it is calibration, and it is the reason the review note asks what you rejected.

Licensing and IP hygiene for generated code



Dependencies it adds

The riskiest licence decision in an agent-assisted diff is usually a new library, not the generated lines. Check what it pulled in, and under what licence, before the diff is merged.



Recognisable code

Ask where a distinctive algorithm or a large verbatim block came from. Anything that looks lifted from a well-known project gets checked against that project's licence.



Attribution and headers

House licence headers, copyright lines and SPDX identifiers belong in CLAUDE.md, so they are applied by default rather than remembered at review time.



What leaves the room

Prompts carry code. Approved, sanitised repositories only — as on Day 1. Whether prompts are retained depends on the configuration: Bedrock model invocation logging is off until someone enables it. Know which setting applies before you promise either way.



Provenance you can show

Keep the branch, the diff and the review note together. "An agent wrote it" is not a defence; "here is what was changed, reviewed and by whom" is a record.



Get a position in writing

Whether generated code is acceptable, and under what conditions, is a question for your legal and engineering leadership — not one a developer should settle per pull request.

Not legal advice — these are the engineering practices that make a legal position enforceable once someone has taken one.

Metrics that matter — and the one that doesn't

Metric	What it tells you	How it misleads
Suggestion acceptance rate	How well the tool is grounded in your repository	Rises when reviewers stop reading. High acceptance with rising rework is the worst combination on this table
Rework / revert rate	Whether accepted work survived contact with the codebase	Lags by days or weeks, so it always looks good at first
Cycle time, ticket to merge	Whether the speed-up reached the thing you sell	Absorbs everything else: review queues, CI time, a release window
Defect escape rate	Whether quality moved — the number that decides this	Needs a stable baseline and enough volume to be more than noise
Review time per diff	Where the cost actually moved	Falls when review gets worse, exactly as it does when review gets easier
Today's number is your own estimate versus your actual time. Report it as estimate-versus-actual, never as a measured productivity gain — an estimate is not a baseline, and this room contains people who know the difference.		

SESSION 7

Debrief & Checkpoint

One reviewed merged change, a review note, and an agreement about what an agent may do unreviewed

16:45 – 17:30



Day 2 recap



Plan before it edits

Plan mode costs a minute and tells you what it believes before it acts on the belief. Rejecting a plan is the cheapest rejection available.



One intent per change

If the diff will not fit on one screen, the slice was too big. This is the only sizing test you need.



Ground the repository, not the laptop

CLAUDE.md, rules, skills and agents are committed and reviewed like code. They outlast the person who wrote them.



Enforce what must not happen

Context is a request the agent weighs. A hook is a gate it cannot argue with. Choose the layer by the cost of being wrong.



Tests before change, in legacy code

Characterisation tests are what let you tell a refactor from a regression — and they are what make the tidier rewrite safe to consider.



Write down what you decided

Accepted, changed, rejected — with evidence for each. "Nothing rejected" is a fair finding when you can say what you checked; it is only a warning sign when you cannot.

End-of-day checkpoint



Each team demonstrates:

- One reviewed, merged agent-assisted change on a branch
- A diff you can walk someone through, line by line
- Tests covering the behaviour you changed
- review-note.md — accepted, changed, rejected, still unverified
- baseline-estimate.md, filled in on both sides

Demo format — 6 teams × 4 min, trainer spot-checks in between

- Walk the diff. Then read out one decision — something you changed or rejected, or something you accepted and can say why.
- Answer in one line: which part of today would have been faster by hand?

Evidence, not a confession

- Say what you checked and how. A note with nothing rejected is fine if it shows the checking; a note with nothing rejected and nothing checked is the rubber-stamping this day exists to prevent. Do not invent an objection to fill the section.

What "merged" means here

- Branch → tests green → /review-diff → review-note.md committed with the change → a second person reads the diff → merge → keep the branch, the note and the run output together as the record.

Stretch (optional)

- Add a PostToolUse hook that runs your formatter on every edited file
- Convert your best command into a skill with a supporting checklist file
- Run /review-diff headlessly with claude -p and have it fail on a non-empty finding list

Before you leave: the team working agreement

One page. What you will, and will not, let an agent do unreviewed in your repository. You revise it on Day 4, once you have seen what goes wrong.

1

Which permission mode is the team's default, and where is that written down?

settings.json, committed

2

What goes in permissions.deny for this repository — today, not eventually?

Secrets, credentials, infrastructure commands

3

What must a human always read before merge, regardless of test results?

Deletions · dependency changes · anything in the security path

4

Which changes may an agent make unreviewed, if any?

Formatting? Test-only diffs? Be specific, or say none

5

What has to be in a review note for a pull request to be acceptable?

At minimum: rejected, and still unverified

6

Who owns the CLAUDE.md, and when does it get pruned?

An unpruned context file costs money on every request

Troubleshooting clinic: the usual suspects

Symptom	Likely cause	Fix
403 naming a model you never chose	An unpinned alias resolved to Claude Code's built-in Bedrock default, which your IAM user cannot invoke. Not a trust problem	/status for the effective model, then set ANTHROPIC_MODEL to your profile ARN. Check the env override, the ARN and your IAM permissions in that order
AccessDeniedException on invoke	Your IAM user can invoke only your own application inference profiles	Set ANTHROPIC_MODEL to the profile ARN on your slip
It ignores an instruction in CLAUDE.md	The file never loaded, or two files contradict each other	/context, check Memory files. Then make the line specific and verifiable
It forgot something mid-session	Said in conversation only, so compaction summarised it away	Anything that must persist belongs in the project-root CLAUDE.md
A rule applies sometimes and not others	It is a path-scoped rule and no matching file has been read since the last compaction	Drop the paths: frontmatter, or move it to CLAUDE.md
/rewind did not undo the damage	The change was made by a Bash command or a subagent — neither is tracked	git. This is why every lab today runs on a branch
The repo's allow-list does not apply	The trust dialog was never accepted — it gates permissions.allow and additionalDirectories only	Run claude interactively in the repo once and accept it
A command is not offered	The file is not in .claude/commands or .claude/skills, or the frontmatter is malformed	Check the path and the frontmatter; restart the session

Ready for Day 3: Tools, MCP, Integration & Security



Docker running

Day 3 runs a local MCP server and a retrieval store in containers. Confirm docker run hello-world works before you leave.



Bring a real integration

One internal system your team would want an agent to reach — a service, a database view, a ticketing API. It becomes your MCP server.



Bring a security question

Whatever worried you today about letting an agent act on your repository. Day 3 answers it against the OWASP LLM Top 10.

Tomorrow

Today the agent worked inside your repository, where the boundary was the working directory. Day 3 gives it your systems — tools you design, an MCP server you build, retrieval you ground it in, and an agent wired into a running application. The boundary stops being a directory and becomes something you have to design.



Questions & reflections

One thing you accepted, changed or rejected today · One thing you'll ground in your own repository tomorrow

Jai Singh

AI Enablement Consultant & Corporate IT Trainer